

Sanity Document Retention — Findings

Project v3sfsmlld · dataset production · caterflow-by-synapse · as of 22 Sept 2026 (updated)

Update notice: this replaces the 21 Sept version. That version's headline finding — "the pipeline is stalled since 20 Sept" — was based on a stale, abandoned MongoDB cluster that production had already migrated away from. Checked directly against the correct production cluster: the archive/cleanup pipeline is running normally (cleanup completing every few hours, archiving every ~5 minutes). The actual reason overdue documents remain in Sanity is different, and is detailed below. The retention-design evidence (orphan scan, file cleanup, deletion order) is unaffected by this correction — it was checked directly against Sanity, not the misidentified cluster.

Summary: the retention design and deletion logic are correct and verified. Documents remain past their 60-day retention cutoff for two compounding, self-resolving reasons, neither of which is a stalled pipeline: (1) many old documents were only copied into the archive database very recently — in one case over 8 months after their business date — and deletion eligibility is gated on that copy date, not the business date; and (2) of the documents that are actually eligible right now, almost all are PurchaseOrders still referenced by a GoodsReceipt that hasn't individually aged out yet, which Sanity's reference integrity correctly blocks. This report documents the evidence for all of this.

TOTAL DOCUMENTS	OVERDUE BY BUSINESS DATE	ELIGIBLE RIGHT NOW
30,948	11,946	558

A note on the 25,000-document figure

An earlier version of this analysis was built around a 25,000-document quota. That number does not match any documented Sanity plan tier and was not independently verified before use. Sanity's actual published tiers (sanity.io/docs/content-lake/technical-limits) are 10,000 documents on the Free plan, 50,000 on Growth, and 1,000,000+ on Enterprise. Since this project holds 30,948 documents and is still actively writing new ones without being blocked, it cannot be on the Free plan (Sanity blocks document creation once that cap is hit) — it must already be on Growth or higher, which would put current usage at roughly 62% of a 50,000 quota, not over any real limit. The exact plan isn't visible via the API token used for this analysis; it should be confirmed on manage.sanity.io before this figure is quoted anywhere else. The retention findings below hold regardless of which tier applies.

Evidence the retention design itself is correct

- **No orphaned documents anywhere.** Every reference field across the entire dataset was scanned — all documents, ~510,000 individual reference fields — and zero point at a non-existent document. Sanity's strong-reference model blocks any delete that would create a dangling reference, and no field in this schema is configured as a weak reference, so this is structurally guaranteed, not just observed.
- **File cleanup is complete and correct.** Every FileAttachment record's parent reference resolves to a live document, and every underlying file asset (image/PDF) is referenced by exactly one FileAttachment — no orphaned metadata records, no unused binary assets left behind.
- **The deletion order is correct.** Collections are processed child-before-parent (FileAttachment/StockSnapshot before GoodsReceipt/DispatchLog/PurchaseOrder/etc.), and a previously identified stale-reference bug (StockSnapshot.lastTransaction blocking parent deletes) was already fixed and verified — zero StockSnapshot documents remain in the dataset today.

What's actually keeping documents around

Checked directly against the correct production archive database (MongoDB cluster cluster0.pdtjsqu.mongodb.net — the 21 Sept report was built against a different, already-abandoned cluster that a prior migration had cut over away from; see the update notice above). The scheduled jobs (cleanup + a 5-minute resume check, per vercel.json) are running normally: the three most recent cleanup runs each completed all 8 collections with zero errors, and archive-to-Mongo runs are completing every ~5 minutes. There is no stuck lock and no stalled run.

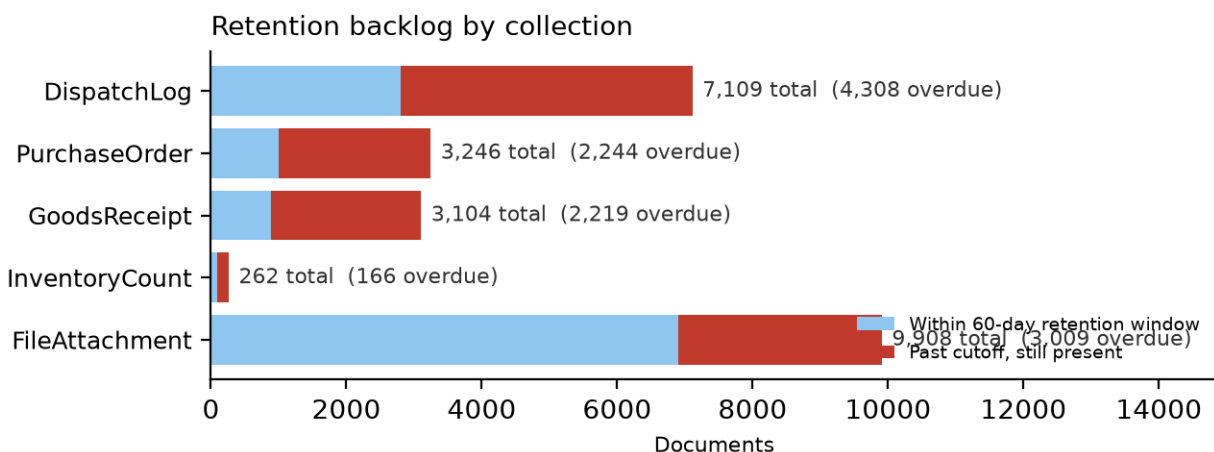
Reason 1 — archival lag delays the deletion clock. A document only becomes eligible for permanent deletion 60 days after it was copied into the archive database (`_archivedAt`), not 60 days after its own business date. Archiving itself was badly backlogged for months — one `PurchaseOrder` with an order date of 10 Dec 2025 wasn't copied into the archive database until 8 Sept 2026, nearly 9 months later — and large batches only caught up on a handful of specific days (1,855 `PurchaseOrders` alone were archived in a single day on 8 Sept). Every document caught in that backlog now has to wait a fresh 60 days from its late archive date, regardless of how overdue it already was by business date. For `PurchaseOrders` alone, 1,684 documents are overdue by business date but were archived too recently to be considered for deletion yet.

Reason 2 — reference-order blocking (expected, not a bug). Of the 558 documents that ARE past the archival-lag gate right now (almost all `PurchaseOrders`), the last several cleanup runs deleted zero of them, with zero errors logged. This is Sanity's own reference-integrity check (`documentHasExistingReferencesError`) silently and correctly skipping each one, because the `GoodsReceipt` that fulfilled it hasn't individually crossed its own cutoff yet — zero `GoodsReceipts` are currently past cutoff. The code already treats this as expected steady-state behavior, not an error, and retries automatically on each pass. It resolves on its own as `GoodsReceipts` age out; forcing these deletions through ahead of that would leave live `GoodsReceipt` documents pointing at `PurchaseOrders` that no longer exist, so this report does not recommend doing that.

Also fixed in code (unrelated to the above, found in the same investigation): the admin "clear lock" endpoint (POST `/api/archive/lock/clear`) only ever reset one of two separate lock singletons — `archive-progress` and `cleanup-progress` are different documents, and only the former could be cleared. Extended the endpoint to accept a target parameter so either (or both) can be reset. Neither lock is actually stuck in production today, so this was preventive, not a fix for the current backlog.

Retention backlog by collection

The direct consequence: documents that should already be gone under the 60-day retention policy (confirmed as the actual production threshold — the default fallback value, since `ARCHIVE_DAYS_THRESHOLD` is not explicitly set in Vercel; the previous report's "90 days" was read from a local `.env` file that had also drifted from what production actually uses) are still present. This is a snapshot of what's overdue by business date right now.



Date range detail

Collection	Total docs	Oldest record	Newest record	Overdue (business date)	% overdue
DispatchLog	7,109	2025-12-08	2026-09-19	4,308	61%
PurchaseOrder	3,246	2025-12-10	2026-09-19	2,244	69%
GoodsReceipt	3,104	2025-12-10	2026-09-19	2,219	71%
InventoryCount	262	2025-12-08	2026-09-19	166	63%
FileAttachment	9,908	2026-06-29	2026-09-19	3,009	30%
TOTAL	23,629			11,946	51%

"Overdue (business date)" counts only records in a finished workflow status (completed/complete/cancelled/adjusted) whose business date is older than the 60-day cutoff (2026-07-24) — i.e. records the system's own rules say are safe to remove. It excludes older records still in draft/pending status, which are correctly retained regardless of age, and excludes one DispatchLog record that carries a corrupted far-future date (a data-entry error, not a retention issue). Of these 11,946, only 558 have also cleared the separate archival-lag gate described above — the rest are correctly retained until their copy into the archive database is itself 60 days old.

Recommended next steps

- No manual intervention needed on the pipeline itself — it's running normally and will keep working through this backlog as GoodsReceipts age out and the archival-lag backlog catches up.
- Worth investigating separately: why PurchaseOrders from December 2025 weren't copied into the archive database until September 2026. If that lag recurs for currently-fresh documents, it will keep re-extending their effective retention well past the intended 60 days.
- Confirm the intended retention period (60 vs. 90 days) with whoever owns this policy, then set `ARCHIVE_DAYS_THRESHOLD` explicitly in Vercel's production environment rather than relying on the code's default — this avoids the exact kind of local/production drift that caused the previous version of this report to look at the wrong cutoff and the wrong database.
- Do not attempt to force-delete PurchaseOrders past Sanity's reference-integrity check — that would leave live GoodsReceipt documents referencing deleted PurchaseOrders.